

MyBudget Personal Finance Manager

C322

Group: 14

Presented by

Conner Inman

Maddox Franklin

Caden Batts

Overview

Supports:

- Multiple accounts
- Recurring transactions
- Customizable budget limits
- Real-time budget alerts

Account	Balance	Limit	Balance Health
Checking	\$2720.00	\$5000.00	54% of limit
Savings	\$9750.00	\$10000.00	97% of limit
Vacation Fund	\$70.00	\$2000.00	3% of limit

MyBudget is a desktop personal finance management application designed to help users take control of their budgets, track transactions, and visualize spending patterns

Observer

The observer pattern is used in conjunction with the command pattern. Every time a command is executed, observers are automatically notified so the notification system isn't constantly checking for changes.

Singleton

The singleton pattern is used to ensure only one active user session at a time as well as one app configuration. Both of these being under the singleton pattern guarantees consistent user information.

Command

The command pattern is used for executing deposits, withdrawals, and setting budgets. It also holds a history of previous commands for being able to undo and redo.

Strategy

The strategy pattern is used to separate the varying report formatting behaviors. The app supports 3 different behaviors, PDF, CSV, and a chart.

Model

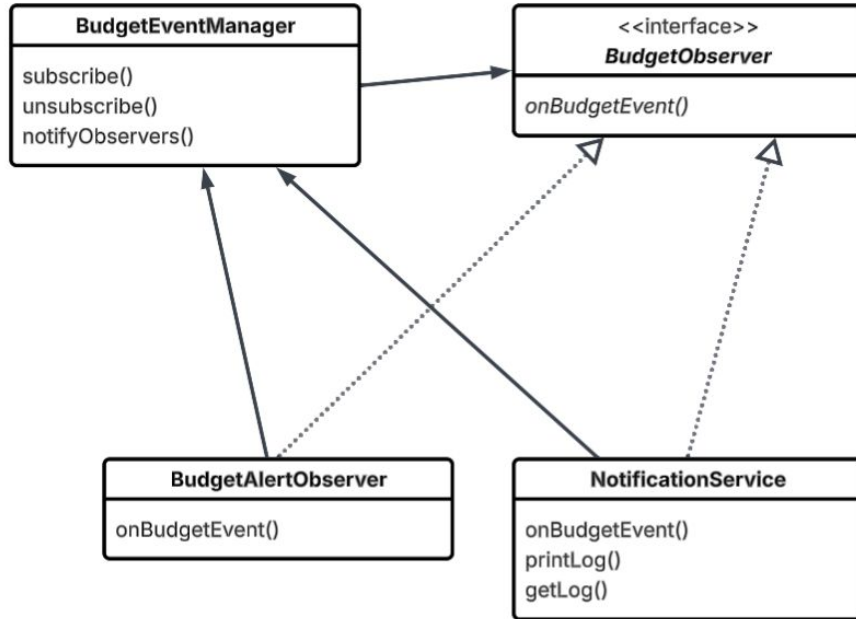
The model is essentially the foundation of our app. It holds the Account and Transaction classes that contain the information used across the app. All the other patterns are built around the model promoting loose coupling.

GUI

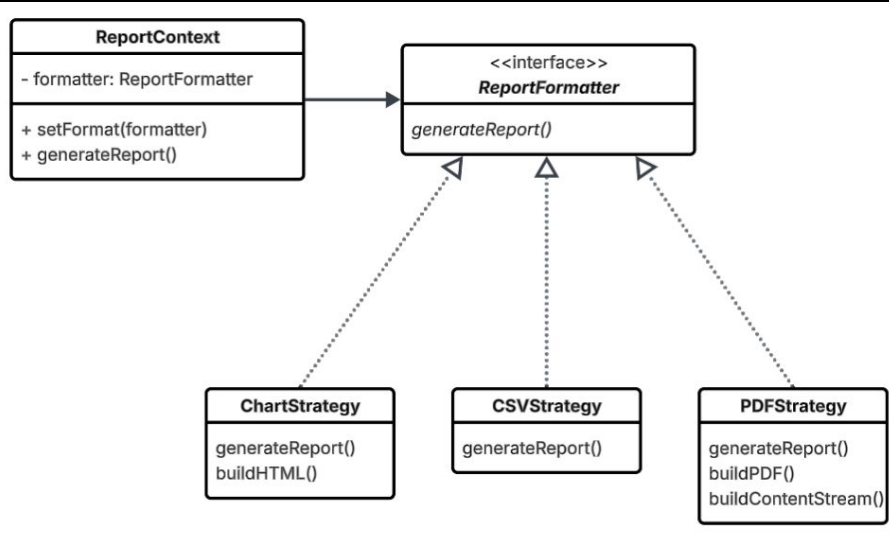
We've also implemented a clean GUI using JavaSwing for a smoother user experience. The GUI unifies the app's features into an easier and more intuitive interface.

Observer Pattern

Used for tracking the budget and spending alert notifications. BudgetEventManager acts as the subject, and everytime a command is executed it notifies the observers, BudgetAlertObserver and NotificationService.

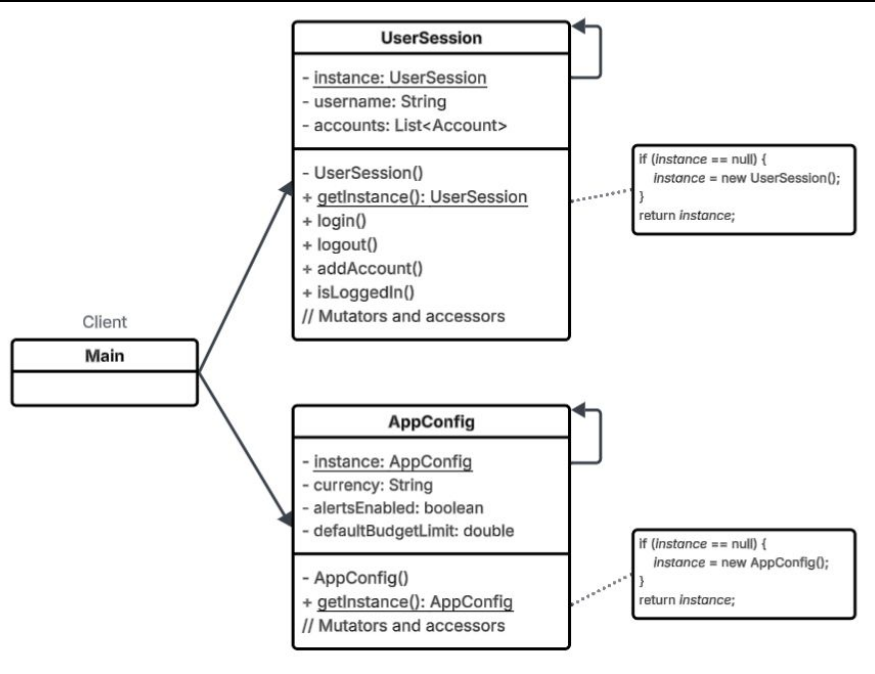


Strategy Pattern



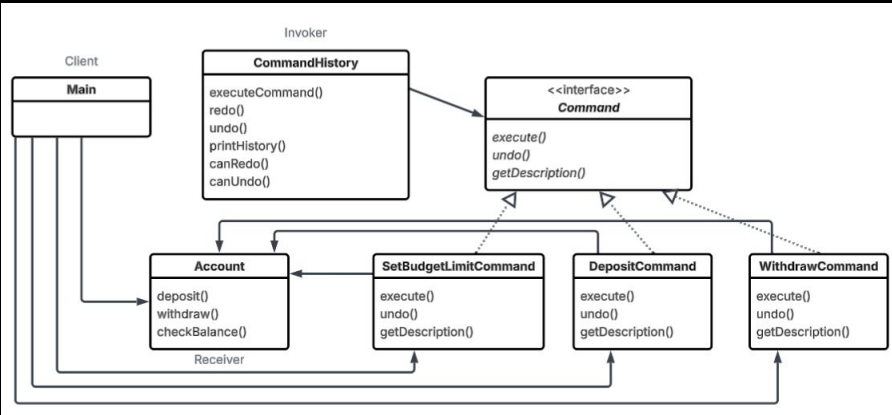
Used for various formatting and file export types. The 3 different report behaviors are separated under the `ReportFormatter` interface and implement their own individual behaviors.

Singleton Pattern



Used to maintain the user's session ensuring only one instance of the application is being used. The 2 singleton instances guarantee consistent data and prevent conflicts.

Command Pattern



Used for executing the functionality of the app. The 3 commands are abstracted away with their own implementations of `execute`, so the invoker can just call `executeCommand` and the corresponding one is used.

Timeline & Contributions

April 14 - 17



Project setup, class diagrams,
Singleton classes, Command base
interface

April 18 - 22



Account model, DepositCommand,
WithdrawCommand,
SetBudgetLimitCommand,
CommandHistory with undo/redo

April 23 - 26



Observer pattern, all three Strategy
implementations, JavaSwing GUI
integration, demo user seeding,
PDF/CSV/Chart output

April 27 - 29



Final report, UML diagrams,
presentation slides, submission

Conner



Overall system architecture, Project
Proposal, develop/debugged code,
Strategy, Command

Maddox



Singleton, Debugger, Presentation,
Code Testing, Command Line
Interface

Caden



Implemented the User Interface: GUI,
Program Demo, Observer alerts

Thank you

MyBudget: A Design Pattern-Driven Personal Finance Application

Conner Inman, Caden Batts, Maddox Franklin

C322: Object-Oriented Software Methods

April 2026

I. Introduction

MyBudget is a desktop personal finance application designed to help users manage accounts, track transactions, and generate financial reports. The system supports real-time balance monitoring, undoable transactions, and multiple reporting formats to encourage informed financial decision-making.

This report presents the system design, implementation, challenges, development timeline, and team contributions, with emphasis on the use of object-oriented design patterns.

II. System Design and Design Patterns

A. Singleton Pattern

The Singleton pattern centralizes shared state through AppConfig and UserSession. AppConfig stores global settings (currency, budget limits, alert preferences), while UserSession manages the active user and accounts.

This approach eliminates the need to pass configuration objects throughout the system and ensures consistent access to application state.

B. Command Pattern

All financial operations are implemented as Command objects with execute() and undo() methods. Key commands include DepositCommand, WithdrawCommand, and SetBudgetLimitCommand.

CommandHistory maintains undo/redo stacks, enabling reversible transactions. Undo operations are delegated to the Account transaction log, ensuring consistency without storing redundant state.

C. Observer Pattern

Budget monitoring is implemented using the Observer pattern. BudgetEventManager evaluates account balances after each transaction and notifies registered observers.

BudgetAlertObserver handles UI alerts, while NotificationService logs events. Alerts are conditionally triggered based on thresholds and global configuration settings.

D. Strategy Pattern

Report generation is implemented using interchangeable strategies via the ReportFormatter interface.

- PDFStrategy generates formatted PDF reports
- CSVStrategy produces structured data output
- ChartStrategy creates interactive HTML dashboards

This design allows runtime selection of output formats without modifying core logic.

E. Pattern Interaction

System functionality relies on coordinated interaction between patterns. User actions trigger Commands, which update account data, notify Observers, and enable reporting via Strategy selection. Shared state is accessed through Singleton instances.

Pattern	Key Classes	Role
Singleton	AppConfig, UserSession	Global config and session state
Command	DepositCommand, WithdrawCommand, SetBudgetLimitCommand, CommandHistory	Undoable transaction history
Observer	BudgetEventManager, BudgetAlertObserver, NotificationService	Real-time budget threshold alerts
Strategy	PDFStrategy, CSVStrategy, ChartStrategy, ReportContext	Pluggable report output formats

Table I: Design Pattern Summary

III. Implementation and Challenges

A. Technology Stack

The system is implemented in Java 17 with a JavaSwing-based GUI. All report formats are generated without external libraries. Compilation and execution are handled via a shell script.

B. GUI Implementation

The JavaSwing interface supports account selection, transaction entry, undo/redo controls, report generation, and notification display.

A defect in account selection for report generation was resolved by refreshing the UI binding, ensuring correct account context.

C. Undo Safety

Undo correctness is ensured by maintaining a transaction log within each Account. Instead of storing snapshots, undo operations reverse the most recent transaction.

This approach guarantees consistency under the LIFO constraint enforced by CommandHistory.

D. Observer–Singleton Coupling

The BudgetEventManager references AppConfig to determine whether alerts are enabled. This integration allows global control of notifications without modifying observer registrations.

IV. Implementation Timeline

Phase	Dates	Deliverables
Phase 1	Apr 14–17	Project setup, class diagrams, Singleton classes, Command base interface
Phase 2	Apr 18–22	Account model, DepositCommand, WithdrawCommand, SetBudgetLimitCommand, CommandHistory with undo/redo
Phase 3	Apr 23–26	Observer pattern, all three Strategy implementations, JavaSwing GUI integration, demo user seeding, PDF/CSV/Chart output
Phase 4	Apr 27–29	Final report, UML diagrams, presentation slides, submission

Table II: Project Timeline

V. Group Member Contributions

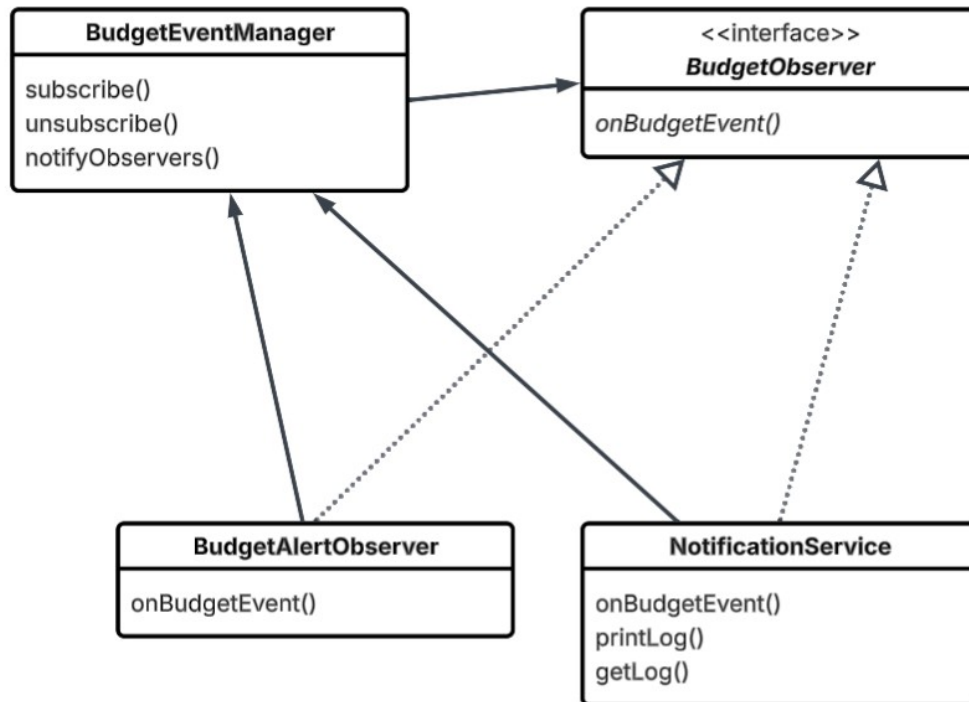
Member	Contributions
Conner Inman	Overall system architecture, Project Proposal, develop/debugged code, Strategy, Command
Maddox Franklin	Singleton, Debugger, Presentation, Code Testing, Command Line Interface
Caden Batts	Implemented the User Interface: GUI, Program Demo, Observer alerts.

Table III: Member Contributions

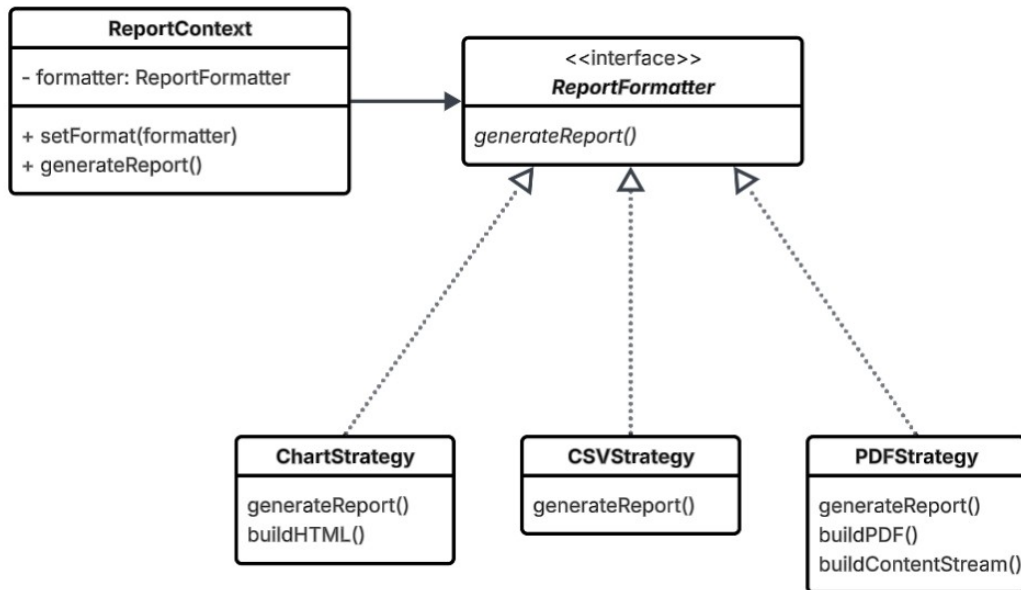
VI. Conclusion

MyBudget successfully demonstrates that design patterns can be composed into a coherent application rather than applied in isolation. The Singleton pattern provided a clean shared-state hub that avoided threading complexity in a single-user context. The Command pattern delivered full undo/redo functionality with minimal bookkeeping inside individual commands. The Observer pattern decoupled budget monitoring from transaction logic. The Strategy pattern enabled three distinct, report formats to be swapped at runtime without modifying any account or command code. Overall, the project met all stated objectives, including the GUI bonus, and demonstrated the practical applicability of design pattern theory in a real-world software context.

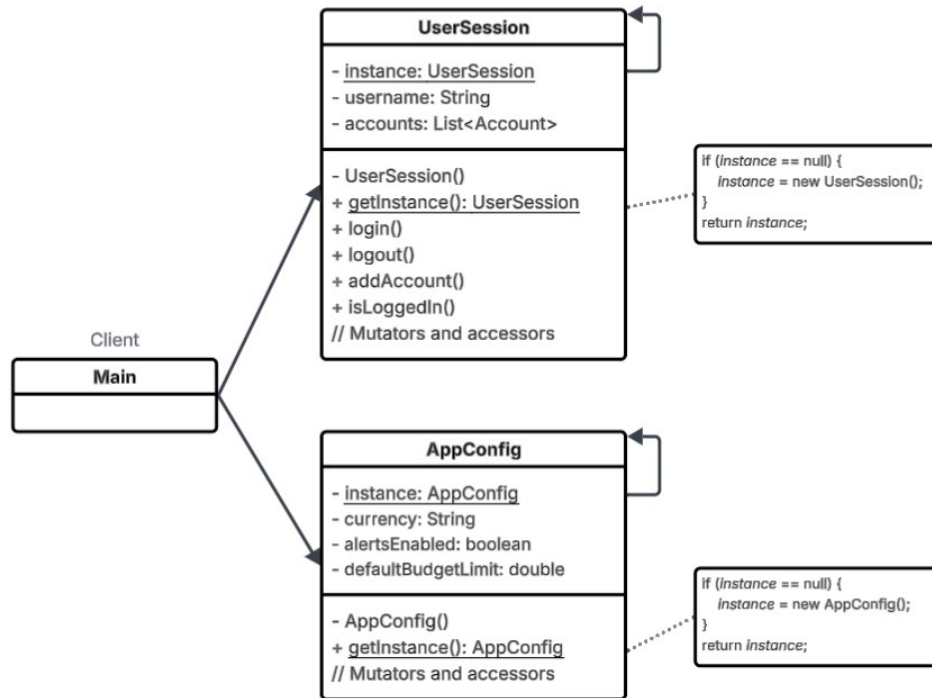
Observer Pattern



Strategy Pattern



Singleton Pattern



Command Pattern

